

Flex/Lex

Thierson Couto Rosa

Instituto de Informática
Universidade Federal de Goiás

1 de março de 2026

Escopo da Aula

- Esta aula tem como objetivo dar informações suficientes para a implementação do analisador léxico do trabalho T1.
- Não é objetivo da aula mostrar todos os recursos da ferramenta.
- Também não é objetivo mostrar como a ferramenta pode ser construída. Esse objetivo será realizado na segunda etapa da disciplina.
- O conteúdo desta aula é baseada no manual do Flex versão 2.6.4.

O que são o Lex e o Flex e o que fazem

- Flex e Lex são programas geradores de analisadores léxicos. O Flex é uma implementação do Lex feita pela GNU. O Lex é um utilitário do Unix.
- Recebem como entrada um arquivo contendo expressões regulares correspondentes aos padrões a serem reconhecidos pelo analisador léxico a ser gerado.
- Geram um arquivo em C contendo o código do analisador léxico gerado.
- O analisador é formado por um conjunto de autômatos finitos. Um autômato é gerado para cada expressão regular.

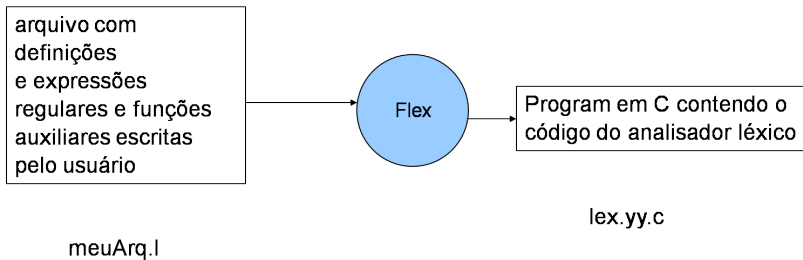
O que são o Lex e o Flex e o que fazem

- Flex e Lex são programas geradores de analisadores léxicos. O Flex é uma implementação do Lex feita pela GNU. O Lex é um utilitário do Unix.
- Recebem como entrada um arquivo contendo expressões regulares correspondentes aos padrões a serem reconhecidos pelo analisador léxico a ser gerado.
- Geram um arquivo em C contendo o código do analisador léxico gerado.
- O analisador é formado por um conjunto de autômatos finitos. Um autômato é gerado para cada expressão regular.
- Como é possível um programa (Flex) gerar um autômato a partir de uma expressão regular?

O que são o Lex e o Flex e o que fazem

- Flex e Lex são programas geradores de analisadores léxicos. O Flex é uma implementação do Lex feita pela GNU. O Lex é um utilitário do Unix.
- Recebem como entrada um arquivo contendo expressões regulares correspondentes aos padrões a serem reconhecidos pelo analisador léxico a ser gerado.
- Geram um arquivo em C contendo o código do analisador léxico gerado.
- O analisador é formado por um conjunto de autômatos finitos. Um autômato é gerado para cada expressão regular.
- Como é possível um programa (Flex) gerar um autômato a partir de uma expressão regular?
- A resposta está na disciplina de Linguagens Formais: *Teorema de Thompson!*

Esquema de Execução do Flex



Seções

Seção de definições

%%

Seção de regras

%%

Seção de código

- O arquivo de entrada para o Flex está dividido em três seções, separadas entre si por uma linha contendo %%.
- O texto do arquivo mistura instruções para o Flex e código escrito na linguagem C. Esses últimos são copiados para o arquivo em linguagem C gerado pelo Flex.

Seção de definições

- Nesta seção são especificadas as definições regulares. Elas possuem o seguinte formato: `nome expressão-regular`, onde `nome` deve iniciar com letra ou `"_"` e pode conter zero ou mais: letra, dígito, `"-"` e `"_"`. A expressão regular inicia após o primeiro caractere não branco (espaço, tabulação) após `nome`. Ex.: `digit [0-9]`.
- Comentários do tipo `/ *...*/` sem indentação são copiados para o arquivo de saída.
- Um texto indentado ou um texto delimitado por `%{` e `%}` é copiado para o arquivo de saída (em C).
- As opções para o comando `flex` podem ser passadas ao programa na linha de comando (ex. `flex -header-file="lexico.h"`) ou podem ser informadas nesta seção através da diretiva `%option`. Por exemplo: `%option header-file="lexico.h"`.
- As declarações de *start conditions* (sc) (explicadas adiante) também ficam nesta seção.

Exemplo

```
1  %{
2  //Inclusao de header files e declaracoes de variaveis
3  #include "lexico.h"
4  #include <stdio.h>
5  #include <stdlib.h>
6  #include <strings.h>
7  int lin_ini_com;
8  %}
9
10 %option yylineno noyywrap
11 /* Definicao regular: DIGIT--> [0-9]. */
12 DIGIT [0-9]
13 /* declaracao de uma start condition */
14 %x comentario
15 /* Fim da secao de definicoes */
```

Definição de uma regra

- Uma regra tem o seguinte formato:

padrão ação
- Não pode haver indentação antes de padrão
- A ação deve aparecer na mesma linha de padrão, separados por pelo menos um espaço ou tabulação.
- Caso uma ação corresponda a várias linhas de código em C, ela deve iniciar com { e terminar com }.
- Se uma definição regular é usada no padrão de uma regra ela deve aparecer entre chaves para indicar que ela deve ser substituída pela expressão regular correspondente. Por exemplo: {DIGIT}+ return NUMERO usa a definição regular DIGIT definida na seção de declarações.

Exemplo

```
1  %%
2  {DIGIT}+    return(NUM);
3  "+"         {return '+';}
4  "*"         {'*'};
5  "{"         {return '{';}
6  "}"         {return '}';}
7  "("         {return '(';}
8  ")"         {return ')';}
9  ";"         {';'};
10 "="         {'='};
11 "<"         {'<'};
12 "<="        {return LEQ;}
13 "while"     {return WHILE;}
14 "if"        {return IF;}
15 "do"        {return DO;}
16 "\n"
17 [ \t]+      ; /* consome caracteres de espaco */
```

Condição de Partida - *Start Condition*

- É um mecanismo que estabelece quais expressões regulares estão ativas durante o processo de análise. Define um contexto no qual um conjunto de expressões regulares é válido.
- São úteis quando várias expressões regulares são necessárias a partir do momento em que uma expressão inicial é encontrada na entrada.
- Ex. Para tratar comentário (*/ * . . . * /*) em C, após encontrar o padrão inicial */ **, seria interessante que o analisador léxico pudesse entrar em um contexto em que outras expressões relevantes para o processamento do comentário pudessem ser utilizadas especificamente para tratar o comentário. Ao término do contexto de comentário, damos uma ordem para que o *parser* volte ao seu contexto normal.

Condição de Partida - Exemplo: comentário

- Na seção de definição (antes do primeiro `%%`) definimos uma condição de partida: `%x comentario`
- Na seção de regras, definimos uma expressão regular que caracteriza o início de um contexto. No caso, a expressão: `"*/"`. Acrescentamos a ação `BEGIN(comentario);`. Regra completa: `"/*" {BEGIN(comentario);}`. Indica que, ao encontrar `/*` na entrada, o *paser* entra no contexto comentário.
- As regras seguintes que devem valer no contexto *c* criado, devem ser escritas na forma `<c>expressão ação`.
- Uma expressão de término do contexto pode ser especificada para marcar o fim de um contexto. A ação: `BEGIN(INITIAL);` é utilizada para indicar ao *parser* para voltar ao contexto normal (denominado `INITIAL`).

Condição de Partida - exemplo completo

```
1
2  "/"      {BEGIN(comentario); lin_ini_com=yylineno;
3            fprintf(stdout, "comentario_ na_ linha_ %d\n",
4            yylineno);
5            }
6  <comentario>[~*\n]*      /*no contexto de comentario, consuma
7            todo caractere diferente de * ou \n */
8  <comentario>"*" + [~/\n]
9  <comentario><<EOF>>      {
10                           fprintf(stderr,
11                           "Erro: _comentario_ da_ linha_ %d_",
12                           lin_ini_com);
13                           fprintf(stderr, "nao_ termina\n");
14                           return(EOF);
15                           }
16  <comentario>\n
17  <comentario>"*" + "/"      {
18                           // volta ao contexto normal do parser
19                           BEGIN(INITIAL);}
```

Condição de Partida

- Há dois tipos de condição de partida: *inclusivas* (declaradas com %s), ou *exclusivas* (declaradas com %x). Entretanto, para o trabalho, é suficiente utilizar as exclusivas, como no exemplo anterior.
- O manual do Flex mostra outras opções de uso para as condições de partida, porém as utilizadas no exemplo anterior são suficientes para o trabalho.
- O exemplo de condição de partida para comentário, pode ser adaptado para processar cadeias de caracteres.

Simplificações de expressões regulares

- No manual do Flex na seção *Patterns* há uma série de simplificações de expressões regulares que podem ajudar na escrita das expressões regulares. Entre elas destacam-se:
 - Classes de caracteres: $[xyz] = x|y|z$, $[\hat{xyz}] = \hat{(x|y|z)}$,
 $[abj - mZ] = (a|b|j|k|l|m|Z)$.
 - Expressões de classes de caracteres: `[:alnum:]`, `[:alpha:]`,
`[:digit:]`, `[:space:]`, `[:lower:]`, `[:upper:]` que devem parecer em
classes de caracteres, i.e. entre `[ e ]`. Exemplo: as expressões a
seguir são equivalentes entre si: `[:alnum:]` = `[:alpha:][:digit:]`
= `[:alpha:][0-9]` = `[a-zA-Z0-9]`

Alguns conceitos importantes em análise léxica

- Vínhamos usando o termo *token* para nos referir a um símbolo terminal da GLC que define a linguagem de programação. Vamos definir melhor os termos que vamos usar:
 - *Token*: representa a linguagem descrita pela expressão regular. O analisador léxico, i.e., a função `yylex()` retorna o valor inteiro correspondente ao token encontrado. Esse retorno é feito em alguma regra. Por exemplo, a regra:
`{DIGIT}+ return(NUMERO);` faz com que valor correspondente ao token NUMERO seja retornado por `yylex()`.
 - *Lexema*: é uma *cadeia* da linguagem regular (*token*) denotada pela expressão regular. Ex. "23", "0", são duas cadeias da linguagem regular NUMERO denotada pela expressão regular `{DIGIT}+`.
 - *token* = linguagem regular e *lexema* = cadeia de um *token*

Alguns conceitos importantes em análise léxica

- Quando o *token* (linguagem regular) é unitário, o *token*, o lexema, e a expressão regular que o descreve possuem a mesma representação. Ex: "[", ")", "while", porque nesse caso a linguagem (*token*) possui um único lexema (única cadeia) e portanto, a expressão regular que denota a linguagem é a própria cadeia.
- Especificamente para o *parser*, o que interessa é qual o *token* (valor inteiro) que `ylex()` retorna.
- Porém, para outras fases do compilador, o lexema também é importante. Por exemplo, na análise semântica é necessário armazenar na tabela de símbolos os lexemas do *token* IDENTIFICADOR, para manter informações de escopo, tipo (int, char, etc) e categoria (variável, parâmetro, função) do lexema específico.

Texto após o segundo %%

- O texto que aparece nessa seção é copiado literalmente para o arquivo de saída na linguagem C gerado pelo Flex (i.e., `lex.yy.c`).
- Esta seção é geralmente formada por funções criadas pelo usuário para chamarem ou serem chamadas por `yylex()`.
- A presença desta seção é opcional. No slide a seguir, um arquivo separado em C (`main.c`) é usado para abrir um arquivo de teste e ficar chamando `yylex()` para encontrar tokens no mesmo. Porém, poderíamos ter colocado a função `main()` que está em `main.c`, nesta seção e teríamos o mesmo efeito.

Exemplo de uso do analisador léxico

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <strings.h>
4  #include "lexico.h"
5  extern char * yytext; //var. global definida pelo Flex
6  extern int yylex(); // funcao analisaor lexico
7  extern int yylineno; // conta as linhas da entrada
8  extern FILE* yyin; //arquivo de entrada
9
10 int main(int argc, char** argv){
11     if(argc!=2)
12         fprintf(stderr, "Uso correto: ./simpleLang_entrada\n");
13     yyin=fopen(argv[1], "r");
14     if(!yyin)
15         {fprintf(stderr, "arquivo nao pode ser aberto\n");
16          return 0;}
17     while(yylex()>0){
18         printf("Lexema: %s na linha %d\n", yytext, yylineno);
19     }
20 }
```

Declarações globais

- O Flex cria algumas declarações globais, além da função `yylex()` para que o código gerado pelo Bison tenha acesso a informações.
 - `char* yytext` - Essa variável contém o lexema encontrado na entrada correspondente a um *token*. Exemplo: se no arquivo de entrada há "345", `yylex()` detecta, essa cadeia através do código gerado para a regra `{DIGIT}+ return(NUMERO);`. Antes de retornar o valor `NUMERO` de *token*, `yylex()` armazena "345" na memória e retorna o ponteiro `yytext` para essa área.
 - `int yylineno`; - Variável global que armazena o número atual da linha do arquivo que está sendo analisado - importante para reportar erro.
 - `FILE* yyin` - Ponteiro para o arquivo que contém o programa que está sendo analisado
 - `int yyleng` - Variável global que contém o tamanho do texto apontado por `yytext`.

Declarações globais

- Quando os arquivos com especificações de entrada para o Bison e para o Flex estão corretos, cada uma das duas ferramentas gera o seu próprio código em linguagem C. Para que possam funcionar precisam ser compiladas separadamente os códigos objetos gerados precisam ser ligados.
- Dado que ambas as ferramentas definem nomes globais (ex. `yyllex()`, `yyltext`, `YYTYPE`, etc), é necessário que a outra ferramenta use a declaração `extern` da linguagem C para poder acessar uma variável global definida pela outra ferramenta. Por exemplo, se o código gerado pelo Bison precisa usar `yyltext`, declarada como global no código gerado pelo Flex, precisa incluir a seguinte declaração: `extern char * yyltext;`

Headerfile usado no exemplo

```
1  #ifndef yyHEADER_H
2  #define yyHEADER_H
3  #define NUM 258
4  #define LEQ 259
5  #define WHILE 260
6  #define IF 262
7  #define DO 263
8  #endif
```

Observação

- Ao encontrar um token unitário correspondente a um caractere (ex. ";", "(", "+"), a ação pode retornar o próprio caractere. Exemplo: ";" return(';'); Neste caso o valor inteiro retornado é o código do caractere na tabela ASCII.
- Ao retornar um token que não corresponde a um caractere, é importante que o valor seja superior ao valor inteiro correspondente ao último símbolo da tabela ASCII, ou seja, retornar um valor ≥ 257 .

Ordem de reconhecimento

- A função `yytext()` ao encontrar um lexema em `yyin` que tem um prefixo aceito por dois autômatos, utiliza aquele autômato que aceita maior quantidade de texto.
- Se dois ou mais autômatos aceitam uma mesma cadeia, aquele que aparece primeiro (correspondente à primeira expressão regular) é escolhido.
 - Logo, as expressões regulares de todas as palavras-chaves devem vir antes da expressão regular para o token identificador, pois a expressão regular para identificador reconhece também todas as palavras-chaves.
 - Se o texto na entrada não é reconhecido por nenhum dos autômatos do *parser*, o próximo caractere na entrada é copiado para a saída padrão na linguagem C.

É necessário indicar se mais de um arquivo deve ser analisado

- O código em C gerado pelo Flex tem o seguinte comportamento:
 - Quando ele encontra um fim de arquivo, ele ativa a função `yywrap`. Se ela retorna zero, é assumido que antes de seu retorno, a função fez `yyin` apontar para um novo arquivo de entrada e o parser continua lendo a partir desse arquivo.
 - Se `yywrap` retorna um valor diferente de zero, o *scanner* `yylex()` termina, retornando 0 para a função que o chamou.
- A princípio, precisaríamos escrever a função `yywrap` com um único comando: `return(1);`, mas o Flex tem opção `nookywrap` que pode ser passada via linha de comando, ou por `%option noyywrap` na seção de definição. Essa opção tem o mesmo efeito, ou seja termina a execução do *parser* ao término do arquivo de entrada.

Exemplo completo para a linguagem SimpleLang

- A função `yytext()` ao encontrar um lexema em `yyin` que tem um prefixo aceito por dois autômatos, utiliza aquele autômato que aceita maior quantidade de texto.
- Se dois ou mais autômatos aceitam uma mesma cadeia, aquele que aparece primeiro (correspondente à primeira expressão regular) é escolhido.
 - Logo, as expressões regulares de todas as palavras-chaves devem vir antes da expressão regular para o token identificador, pois a expressão regular para identificador reconhece também todas as palavras-chaves.
 - Se o texto na entrada não é reconhecido por nenhum dos autômatos do *parser*, o próximo caractere na entrada é copiado para a saída padrão na linguagem C.